

GPT2 is in pytorch_transformers

```
In [1]: import sys
# You may need to play around with the path like I did to find packages
!{sys.executable} -m pip install numpy
!{sys.executable} -m pip install scipy
!{sys.executable} -m pip install pytorch_transformers
```

```

Collecting numpy
  Using cached numpy-2.0.2-cp39-cp39-macosx_14_0_arm64.whl.metadata (60 k
B)
Using cached numpy-2.0.2-cp39-cp39-macosx_14_0_arm64.whl (5.3 MB)
Installing collected packages: numpy
Successfully installed numpy-2.0.2
Collecting scipy
  Using cached scipy-1.13.1-cp39-cp39-macosx_12_0_arm64.whl.metadata (60 k
B)
Requirement already satisfied: numpy<2.3,>=1.22.4 in ./venv/lib/python3.
9/site-packages (from scipy) (2.0.2)
Using cached scipy-1.13.1-cp39-cp39-macosx_12_0_arm64.whl (30.3 MB)
Installing collected packages: scipy
Successfully installed scipy-1.13.1
Collecting pytorch_transformers
  Using cached pytorch_transformers-1.2.0-py3-none-any.whl.metadata (21 k
B)
Collecting torch>=1.0.0 (from pytorch_transformers)
  Using cached torch-2.5.1-cp39-none-macosx_11_0_arm64.whl.metadata (28 k
B)
Requirement already satisfied: numpy in ./venv/lib/python3.9/site-package
s (from pytorch_transformers) (2.0.2)
Collecting boto3 (from pytorch_transformers)
  Using cached boto3-1.35.97-py3-none-any.whl.metadata (6.7 kB)
Collecting requests (from pytorch_transformers)
  Using cached requests-2.32.3-py3-none-any.whl.metadata (4.6 kB)
Collecting tqdm (from pytorch_transformers)
  Using cached tqdm-4.67.1-py3-none-any.whl.metadata (57 kB)
Collecting regex (from pytorch_transformers)
  Using cached regex-2024.11.6-cp39-cp39-macosx_11_0_arm64.whl.metadata (4
0 kB)
Collecting sentencepiece (from pytorch_transformers)
  Using cached sentencepiece-0.2.0-cp39-cp39-macosx_11_0_arm64.whl.metadat
a (7.7 kB)
Collecting sacremoses (from pytorch_transformers)
  Using cached sacremoses-0.1.1-py3-none-any.whl.metadata (8.3 kB)
Collecting filelock (from torch>=1.0.0->pytorch_transformers)
  Using cached filelock-3.16.1-py3-none-any.whl.metadata (2.9 kB)
Requirement already satisfied: typing-extensions>=4.8.0 in ./venv/lib/pyt
hon3.9/site-packages (from torch>=1.0.0->pytorch_transformers) (4.12.2)
Collecting networkx (from torch>=1.0.0->pytorch_transformers)
  Using cached networkx-3.2.1-py3-none-any.whl.metadata (5.2 kB)
Collecting jinja2 (from torch>=1.0.0->pytorch_transformers)
  Using cached jinja2-3.1.5-py3-none-any.whl.metadata (2.6 kB)
Collecting fsspec (from torch>=1.0.0->pytorch_transformers)
  Using cached fsspec-2024.12.0-py3-none-any.whl.metadata (11 kB)
Collecting sympy==1.13.1 (from torch>=1.0.0->pytorch_transformers)
  Using cached sympy-1.13.1-py3-none-any.whl.metadata (12 kB)
Collecting mpmath<1.4,>=1.1.0 (from sympy==1.13.1->torch>=1.0.0->pytorch_t
ransformers)
  Using cached mpmath-1.3.0-py3-none-any.whl.metadata (8.6 kB)
Collecting botocore<1.36.0,>=1.35.97 (from boto3->pytorch_transformers)
  Using cached botocore-1.35.97-py3-none-any.whl.metadata (5.7 kB)
Collecting jmespath<2.0.0,>=0.7.1 (from boto3->pytorch_transformers)
  Using cached jmespath-1.0.1-py3-none-any.whl.metadata (7.6 kB)
Collecting s3transfer<0.11.0,>=0.10.0 (from boto3->pytorch_transformers)
  Using cached s3transfer-0.10.4-py3-none-any.whl.metadata (1.7 kB)
Collecting charset-normalizer<4,>=2 (from requests->pytorch_transformers)
  Using cached charset_normalizer-3.4.1-cp39-cp39-macosx_10_9_universal2.w
hl.metadata (35 kB)

```

```

Collecting idna<4,>=2.5 (from requests->pytorch_transformers)
  Using cached idna-3.10-py3-none-any.whl.metadata (10 kB)
Collecting urllib3<3,>=1.21.1 (from requests->pytorch_transformers)
  Using cached urllib3-2.3.0-py3-none-any.whl.metadata (6.5 kB)
Collecting certifi>=2017.4.17 (from requests->pytorch_transformers)
  Using cached certifi-2024.12.14-py3-none-any.whl.metadata (2.3 kB)
Collecting click (from sacremoses->pytorch_transformers)
  Using cached click-8.1.8-py3-none-any.whl.metadata (2.3 kB)
Collecting joblib (from sacremoses->pytorch_transformers)
  Using cached joblib-1.4.2-py3-none-any.whl.metadata (5.4 kB)
Requirement already satisfied: python-dateutil<3.0.0,>=2.1 in ./venv/lib/
python3.9/site-packages (from botocore<1.36.0,>=1.35.97->boto3->pytorch_tr
ansformers) (2.9.0.post0)
Collecting urllib3<3,>=1.21.1 (from requests->pytorch_transformers)
  Using cached urllib3-1.26.20-py2.py3-none-any.whl.metadata (50 kB)
Collecting MarkupSafe>=2.0 (from jinja2->torch>=1.0.0->pytorch_transformer
s)
  Using cached MarkupSafe-3.0.2-cp39-cp39-macosx_11_0_arm64.whl.metadata
(4.0 kB)
Requirement already satisfied: six>=1.5 in ./venv/lib/python3.9/site-pack
ages (from python-dateutil<3.0.0,>=2.1->botocore<1.36.0,>=1.35.97->boto3->
pytorch_transformers) (1.17.0)
Using cached pytorch_transformers-1.2.0-py3-none-any.whl (176 kB)
Using cached torch-2.5.1-cp39-none-macosx_11_0_arm64.whl (63.9 MB)
Using cached sympy-1.13.1-py3-none-any.whl (6.2 MB)
Using cached boto3-1.35.97-py3-none-any.whl (139 kB)
Using cached regex-2024.11.6-cp39-cp39-macosx_11_0_arm64.whl (284 kB)
Using cached requests-2.32.3-py3-none-any.whl (64 kB)
Using cached sacremoses-0.1.1-py3-none-any.whl (897 kB)
Using cached sentencepiece-0.2.0-cp39-cp39-macosx_11_0_arm64.whl (1.2 MB)
Using cached tqdm-4.67.1-py3-none-any.whl (78 kB)
Using cached botocore-1.35.97-py3-none-any.whl (13.3 MB)
Using cached certifi-2024.12.14-py3-none-any.whl (164 kB)
Using cached charset_normalizer-3.4.1-cp39-cp39-macosx_10_9_universal2.whl
(197 kB)
Using cached idna-3.10-py3-none-any.whl (70 kB)
Using cached jmespath-1.0.1-py3-none-any.whl (20 kB)
Using cached s3transfer-0.10.4-py3-none-any.whl (83 kB)
Using cached urllib3-1.26.20-py2.py3-none-any.whl (144 kB)
Using cached click-8.1.8-py3-none-any.whl (98 kB)
Using cached filelock-3.16.1-py3-none-any.whl (16 kB)
Using cached fsspec-2024.12.0-py3-none-any.whl (183 kB)
Using cached jinja2-3.1.5-py3-none-any.whl (134 kB)
Using cached joblib-1.4.2-py3-none-any.whl (301 kB)
Using cached networkx-3.2.1-py3-none-any.whl (1.6 MB)
Using cached MarkupSafe-3.0.2-cp39-cp39-macosx_11_0_arm64.whl (12 kB)
Using cached mpmath-1.3.0-py3-none-any.whl (536 kB)
Installing collected packages: sentencepiece, mpmath, urllib3, tqdm, symp
y, regex, networkx, MarkupSafe, joblib, jmespath, idna, fsspec, filelock,
click, charset-normalizer, certifi, sacremoses, requests, jinja2, botocor
e, torch, s3transfer, boto3, pytorch_transformers
Successfully installed MarkupSafe-3.0.2 boto3-1.35.97 botocore-1.35.97 cer
tifi-2024.12.14 charset-normalizer-3.4.1 click-8.1.8 filelock-3.16.1 fsspe
c-2024.12.0 idna-3.10 jinja2-3.1.5 jmespath-1.0.1 joblib-1.4.2 mpmath-1.3.
0 networkx-3.2.1 pytorch_transformers-1.2.0 regex-2024.11.6 requests-2.32.
3 s3transfer-0.10.4 sacremoses-0.1.1 sentencepiece-0.2.0 sympy-1.13.1 torc
h-2.5.1 tqdm-4.67.1 urllib3-1.26.20

```

Import required libraries

```
In [2]: print(sys.path)
import torch
from pytorch_transformers import GPT2Tokenizer, GPT2LMHeadModel
```

```
['/Library/Developer/CommandLineTools/Library/Frameworks/Python3.framework/Versions/3.9/lib/python3.9.zip', '/Library/Developer/CommandLineTools/Library/Frameworks/Python3.framework/Versions/3.9/lib/python3.9', '/Library/Developer/CommandLineTools/Library/Frameworks/Python3.framework/Versions/3.9/lib/python3.9/lib-dynload', '', '/Users/eamin/Desktop/data/学习资料/2025_spring/GenAI/Lec1/Lec1_HW_YimingCheng_124505888/HW1-1/.venv/lib/python3.9/site-packages']
```

Load pre-trained model tokenizer (vocabulary)

```
In [3]: tokenizer = GPT2Tokenizer.from_pretrained('gpt2')
```

```
100%|██████████| 1042301/1042301 [00:00<00:00, 6451562.67B/s]
100%|██████████| 456318/456318 [00:00<00:00, 3519967.21B/s]
```

Encode the sentence to complete

```
In [4]: text = "The answer to the universe is not to be found in the"
indexed_tokens = tokenizer.encode(text)
```

Convert indexed tokens in a PyTorch tensor

```
In [5]: tokens_tensor = torch.tensor([indexed_tokens])
```

Load pre-trained model (weights)

```
In [6]: model = GPT2LMHeadModel.from_pretrained('gpt2')
```

```
100%|██████████| 665/665 [00:00<00:00, 1359927.92B/s]
100%|██████████| 548118077/548118077 [02:10<00:00, 4212157.50B/s]
/Users/eamin/Desktop/data/学习资料/2025_spring/GenAI/Lec1/Lec1_HW_YimingCheng_124505888/HW1-1/.venv/lib/python3.9/site-packages/pytorch_transformers/modeling_utils.py:539: FutureWarning: You are using `torch.load` with `weights_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for `weights_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
state_dict = torch.load(resolved_archive_file, map_location='cpu')
```

Set the model in evaluation mode to deactivate the DropOut modules

```
In [7]: model.eval()
```

```

Out [7]: GPT2LMHeadModel(
  (transformer): GPT2Model(
    (wte): Embedding(50257, 768)
    (wpe): Embedding(1024, 768)
    (drop): Dropout(p=0.1, inplace=False)
    (h): ModuleList(
      (0-11): 12 x Block(
        (ln_1): LayerNorm((768,)), eps=1e-05, elementwise_affine=True)
        (attn): Attention(
          (c_attn): Conv1D()
          (c_proj): Conv1D()
          (attn_dropout): Dropout(p=0.1, inplace=False)
          (resid_dropout): Dropout(p=0.1, inplace=False)
        )
        (ln_2): LayerNorm((768,)), eps=1e-05, elementwise_affine=True)
        (mlp): MLP(
          (c_fc): Conv1D()
          (c_proj): Conv1D()
          (dropout): Dropout(p=0.1, inplace=False)
        )
      )
    )
    (ln_f): LayerNorm((768,)), eps=1e-05, elementwise_affine=True)
  )
  (lm_head): Linear(in_features=768, out_features=50257, bias=False)
)

```

If you have a GPU, uncomment the following to put everything on cuda

```

In [9]: # tokens_tensor = tokens_tensor.to('cuda')
        # model.to('cuda')

        tokens_tensor = tokens_tensor.to('mps')
        model.to('mps')

```

```

Out [9]: GPT2LMHeadModel(
  (transformer): GPT2Model(
    (wte): Embedding(50257, 768)
    (wpe): Embedding(1024, 768)
    (drop): Dropout(p=0.1, inplace=False)
    (h): ModuleList(
      (0-11): 12 x Block(
        (ln_1): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
        (attn): Attention(
          (c_attn): Conv1D()
          (c_proj): Conv1D()
          (attn_dropout): Dropout(p=0.1, inplace=False)
          (resid_dropout): Dropout(p=0.1, inplace=False)
        )
        (ln_2): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
        (mlp): MLP(
          (c_fc): Conv1D()
          (c_proj): Conv1D()
          (dropout): Dropout(p=0.1, inplace=False)
        )
      )
    )
    (ln_f): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
  )
  (lm_head): Linear(in_features=768, out_features=50257, bias=False)
)

```

Predict all tokens

```

In [10]: with torch.no_grad():
          outputs = model(tokens_tensor)
          predictions = outputs[0]

```

Get the predicted next sub-word

```

In [11]: predicted_index = torch.argmax(predictions[0, -1, :]).item()
          predicted_text = tokenizer.decode(indexed_tokens + [predicted_index])

```

Print the predicted word

```

In [12]: print(predicted_text)

```

The answer to the universe is not to be found in the universe

```

In [14]: # Extract probabilities for the next token
          next_token_logits = predictions[0, -1, :]
          probabilities = torch.softmax(next_token_logits, dim=-1)

          # Decode all tokens and probabilities
          top_k = 10 # Number of top tokens to display
          sorted_indices = torch.argsort(probabilities, descending=True)

          # Print the top tokens and their probabilities
          print("Top tokens excluding 'universe':")
          for idx in sorted_indices[:top_k]:
              token = tokenizer.decode([idx.item()]).strip()
              if token != "universe": # Exclude the token "universe"
                  print(f"Token: '{token}', Probability: {probabilities[idx].item():.4f}")

```

Top tokens excluding 'universe':
 Token: 'Bible', Probability: 0.0311
 Token: 'laws', Probability: 0.0157
 Token: 'physical', Probability: 0.0127
 Token: '', Probability: 0.0103
 Token: 'cosmos', Probability: 0.0082
 Token: 'form', Probability: 0.0079
 Token: 'first', Probability: 0.0074
 Token: 'very', Probability: 0.0065
 Token: 'simple', Probability: 0.0060

```
In [17]: # Generate text of varying lengths
device = torch.device("mps")
# Function to generate text with a specified number of tokens
def generate_text(input_text, num_tokens, model, tokenizer, device):
    indexed_tokens = tokenizer.encode(input_text)
    tokens_tensor = torch.tensor([indexed_tokens]).to(device)

    model.eval()
    with torch.no_grad():
        for _ in range(num_tokens):
            outputs = model(tokens_tensor)
            predictions = outputs[0]
            predicted_index = torch.argmax(predictions[0, -1, :]).item()
            indexed_tokens.append(predicted_index)
            tokens_tensor = torch.tensor([indexed_tokens]).to(device)

    return tokenizer.decode(indexed_tokens)

# Input text
input_text = "The answer to the universe is not to be found in the"

# Generate texts of different lengths
lengths = [1, 5, 10, 20]
for length in lengths:
    generated_text = generate_text(input_text, length, model, tokenizer,
    print(f"\nGenerated text ({length} tokens):\n{generated_text}")
```

Generated text (1 tokens):

The answer to the universe is not to be found in the universe

Generated text (5 tokens):

The answer to the universe is not to be found in the universe itself, but in

Generated text (10 tokens):

The answer to the universe is not to be found in the universe itself, but in the universe itself.

Generated text (20 tokens):

The answer to the universe is not to be found in the universe itself, but in the universe itself.

The universe is not a place of infinite numbers

```
In [18]: # Generate text for a single role and query
role = "scientist"
query = "explain the concept of time."
```

```
# Combine role and query
input_text = f"As a {role}, {query}"
indexed_tokens = tokenizer.encode(input_text)
tokens_tensor = torch.tensor([indexed_tokens]).to(device)

# Generate text
num_tokens = 20
with torch.no_grad():
    for _ in range(num_tokens):
        outputs = model(tokens_tensor)
        predictions = outputs[0]
        predicted_index = torch.argmax(predictions[0, -1, :]).item()
        indexed_tokens.append(predicted_index)
        tokens_tensor = torch.tensor([indexed_tokens]).to(device)

# Decode and print
generated_text = tokenizer.decode(indexed_tokens)
print(f"Role: {role}, Query: {query}\nGenerated Text:\n{generated_text}")
```

Role: scientist, Query: explain the concept of time.

Generated Text:

As a scientist, explain the concept of time.

The concept of time is a concept that is often used to describe the time of a person